CYBERSECURITY GUIDE BOOK

CHAPTER 3

Programming Basics

Roadmap step 3 of 12

Master the fundamentals first, then specialize.
Every concept is shown from two sides: how an attacker abuses it, and how a defender (SOC analyst) detects and stops it.

Author: ashihh07
For learning and authorized lab use only.

> *You do not need to become a software engineer. You need to read code, write small tools, and spot the coding mistakes that attackers exploit. This chapter teaches exactly that.*

About this chapter

A SOC analyst who can script turns a four-hour log review into a four-second command. A pentester who can read code finds the bug the scanner missed. Malware, exploits, detection rules and security tools are all code. In Chapter 2 you chained commands with pipes. Now you learn to put logic around them.

The roadmap lists four items. Each gets a security-focused section:

- **Python (core):** your main tool for automation, parsing and custom utilities.
- **Bash scripting:** glue for the Linux command line, everywhere on servers.
- **JavaScript (basics):** the language of the browser, and the language behind XSS.
- **Data formats (JSON, XML):** how tools, APIs and logs exchange information.

How each topic is built

- **Concept**: the minimum you need, in plain language.
- **Attacker view**: how the same code or feature gets abused (for understanding and authorized testing only).
- **Defender view**: how to write it safely and how to detect misuse.
- **Labs**: small programs you run on your own VM.

Contents

#	Section	Roadmap item covered
1	Programming for security work	Orientation
2	Python core	Python (Core)
3	Bash scripting	Bash Scripting
4	JavaScript basics	JavaScript (Basics)
5	Data formats: JSON and XML	Data Formats (JSON, XML)
6	Secure coding rules in one table	All of the above
7	Hands-on labs	Practice for all of the above
8	Quick revision sheet	Syntax cheat sheets
9	Self-check quiz and answers	Test yourself

Ground rule for every lab in this book

- Only scan, capture or test systems you own or have written permission to test.
- Run every script in a VM you own. Read a script before you run it, especially with `sudo`.
- Unauthorized access can be illegal even if you mean no harm.

1. Programming for security work

Concept

You will write four kinds of programs again and again:

Kind	Example	Best language
Log and data processing	Count failed logins per IP, parse a scan result	Python, Bash with `awk`
Automation and glue	Run a scan, save output, email a summary, schedule it with cron	Bash, Python
Small network tools	Check which ports are open, test HTTP headers	Python
Reading browser-side code	Understand why a page is vulnerable to XSS	JavaScript

How to learn it

1. Type every example. Do not paste. Typing forces you to read.
2. Change one thing, predict the result, run it, compare.
3. Make a small tool that solves a real problem you have, such as Lab 3's log parser.
4. Read other people's short scripts in security repos and explain each line to yourself.

2. Python core

Concept

Python is readable, ships with Linux, and has a library for almost everything. Check your version and set up a **virtual environment** so project packages stay separate:

```
python3 --version
sudo apt install python3-venv python3-pip     # Debian family, if missing
python3 -m venv venv                          # create an isolated environment
source venv/bin/activate                      # prompt shows (venv)
pip install requests                          # installs only inside this venv
deactivate                                    # leave the venv
```

> On newer Ubuntu and Debian, `pip install` outside a venv fails with `externally-managed-environment`. That is intentional: it protects system tools. Use a venv.

The building blocks

```
# Variables and types
name = "alice"            # str
port = 22                 # int
ratio = 0.75              # float
is_open = True            # bool
ports = [22, 80, 443]     # list: ordered, changeable
point = (10, 20)          # tuple: ordered, fixed
seen = {"10.0.0.1", "10.0.0.2"}   # set: unique items only
user = {"name": "alice", "uid": 1001}   # dict: key -> value

# Strings
line = "Failed password for root from 203.0.113.5 port 4222 ssh2"
parts = line.split()      # split on spaces -> list of words
print(parts[-4])          # 203.0.113.5  (negative index counts from the end)
print(f"{name} uses port {port}")   # f-string formatting

# Decisions and loops
if port == 22:
    print("SSH")
elif port in (80, 443):
    print("web")
else:
    print("other")

for p in ports:
    print(p)

# Functions
def is_private(ip):
    return ip.startswith(("10.", "192.168."))

print(is_private("192.168.1.5"))   # True
```

Indentation (4 spaces) is part of the syntax. It defines which lines belong to an `if`, `for` or `def`.

Files, errors and modules

```
try:
    with open("/var/log/auth.log") as f:       # 'with' closes the file for you
        for line in f:
            if "Failed password" in line:
                print(line.strip())
except FileNotFoundError:
    print("log file not found")
except PermissionError:
    print("run with sudo")
```

- `import module` pulls in code: `os`, `sys`, `re`, `json`, `socket`, `hashlib`, `subprocess`, `ipaddress`, `collections` are all built in.
- `pip install name` adds third-party modules such as `requests`.

Four security-flavoured programs

1. Port checker. Use it on `127.0.0.1` or machines you own.

```
import socket

def check_port(host, port, timeout=1.0):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.settimeout(timeout)
        return s.connect_ex((host, port)) == 0   # 0 means the TCP handshake worked

for p in (22, 80, 443):
    print(p, "open" if check_port("127.0.0.1", p) else "closed")
```

This is the Chapter 1 handshake in six lines. `connect_ex` returns 0 when SYN, SYN-ACK, ACK completes.

2. Failed-login counter. Regular expressions (`re`) find patterns, `Counter` counts them.

```
import re
from collections import Counter

pattern = re.compile(r"Failed password for (?:invalid user )?(\S+) from (\d+\.\d+\.\d+\.
\d+)")
counts = Counter()

with open("/var/log/auth.log") as f:
    for line in f:
        m = pattern.search(line)
        if m:
            counts[m.group(2)] += 1          # group(2) is the IP

for ip, n in counts.most_common(5):
    print(f"{n:5}  {ip}")
```

Run it with `sudo python3 failed_ssh.py`. It is the Chapter 2 pipeline, but now you can add rules: ignore your own IP, alert above a threshold, write a report.

3. File integrity hash.

```
import hashlib, sys

def sha256(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b""):   # read in pieces, any file size
            h.update(chunk)
    return h.hexdigest()

print(sha256(sys.argv[1]))     # python3 hash.py ubuntu.iso
```

Compare the output with the checksum the project publishes. The same idea powers file-integrity monitoring and malware hash lookups.

4. Check security headers (only on sites you own or may test).

```
import requests

r = requests.get("https://example.com", timeout=5)
for h in ("Strict-Transport-Security", "Content-Security-Policy", "X-Content-Type-Options"):
    print(f"{h}: {r.headers.get(h, 'MISSING')}")
```

This automates the header table from Chapter 1.

Attacker view and defender view

Attacker view: command injection

If code builds a shell command by gluing in user input, the user can add their own commands:

```
import os
host = input("Host to ping: ")
os.system("ping -c 1 " + host)        # input "8.8.8.8; cat /etc/passwd" runs BOTH
commands
```

The shell treats `;`, `&&`, `|` and backticks as command separators. Any program that passes unchecked text to a shell is a target.

Defender view: write it safely

```
import subprocess, ipaddress

host = input("Host to ping: ")
try:
    ip = ipaddress.ip_address(host)                      # rejects anything that is
not an IP
except ValueError:
    raise SystemExit("not a valid IP address")
subprocess.run(["ping", "-c", "1", str(ip)], check=False)  # list form: no shell
involved
```

- Validate input against what you expect (allow-list), not against what you fear.
- Use `subprocess.run([...])` with a list and no `shell=True`.
- Never call `eval()` or `exec()` on data you did not write. Do not load untrusted data with `pickle`; use `yaml.safe_load`, not `yaml.load`.
- Keep secrets out of source code: read them from environment variables (`os.environ["API_KEY"]`).

3. Bash scripting

Concept

A Bash script is a text file of commands plus logic. Use Bash when the job is "run these programs in order and process their text output". Switch to Python when the logic gets complex.

```
#!/bin/bash
# first line (shebang) says which interpreter runs the file
name="alice"
echo "Hello, $name"          # double quotes expand variables
echo 'No $expansion here'    # single quotes do not
today=$(date +%F)            # command substitution: capture output
echo "Report for $today"
```

```
chmod +x script.sh
./script.sh arg1 arg2
```

Arguments, tests and loops

Item	Meaning
`$0` `$1` `$2`	Script name, first argument, second argument
`$#`	Number of arguments
`$@`	All arguments
`$?`	Exit code of the last command (0 = success)
`[[ -f f ]]` / `[[ -d d ]]`	File exists / directory exists
`[[ -z s ]]` / `[[ -n s ]]`	String is empty / not empty
`-eq -ne -lt -gt`	Number comparisons (`[[ $n -gt 5 ]]`)
`==` `!=`	String comparisons inside `[[ ]]`

```
#!/bin/bash
# usage: ./check_log.sh /var/log/auth.log
if [[ $# -ne 1 ]]; then
    echo "usage: $0 logfile" >&2
    exit 1
fi

log="$1"
if [[ ! -f "$log" ]]; then
    echo "no such file: $log" >&2
    exit 2
fi

count=$(grep -c "Failed password" "$log")
echo "Failed logins: $count"

if [[ $count -gt 20 ]]; then
    echo "ALERT: possible brute force"
fi
```

Loops:

```
for ip in 10.0.0.5 10.0.0.6 10.0.0.7; do
    ping -c 1 -W 1 "$ip" > /dev/null && echo "$ip up"
done

while read -r line; do                # read a file line by line
    echo "$line"
done < ips.txt
```

Functions and safer scripts

```
#!/bin/bash
set -euo pipefail          # stop on error, on unset variable, and on failed pipe stage

log() { echo "[$(date +%T)] $*"; }

dir="${1:?usage: $0 directory}"      # fail with a message if no argument given
log "cleaning $dir"
rm -rf "${dir:?}/tmp"                # ${dir:?} refuses to run if dir is empty
```

An unquoted empty variable turns `rm -rf $dir/` into `rm -rf /`. Quote every variable and use `${var:?}` before destructive commands.

A small tool: ping sweep of your own lab network

```
#!/bin/bash
# usage: ./sweep.sh 192.168.56        (your own host-only network)
prefix="${1:?usage: $0 first-three-octets}"
for i in $(seq 1 254); do
    ping -c 1 -W 1 "$prefix.$i" > /dev/null 2>&1 && echo "$prefix.$i is up" &
done
wait
```

The `&` runs each ping in the background so 254 hosts finish in about a second. `wait` pauses until all are done.

Scheduling

Put a script in cron (Chapter 2) to run it automatically:
`0 6 * * * /home/alice/failed-ssh.sh >> /home/alice/report.txt`. Use full paths in cron.

Attacker view

- Bash is the attacker's favourite glue too: download, make executable, run, schedule. A script in `/tmp` plus a cron line is a classic persistence pattern.
- Unquoted variables and `eval` in an admin's script can be abused the same way as command injection in Python.
- Reverse shells are one-line Bash commands. You will recognise them in logs once you can read Bash.

Defender view

- Quote variables, use `set -euo pipefail`, and run `shellcheck script.sh` (`sudo apt install shellcheck`) to catch common mistakes.
- Never run `curl ... | bash` from an untrusted source. Download, read, run.
- Scripts that run as root must be owned by root and not writable by others (Chapter 2 permissions).
- In incident response, a Bash script that gathers `ps`, `ss`, `crontab`, `last` and file hashes into one report is a standard first tool.

4. JavaScript basics

Concept

JavaScript (JS) runs inside every browser, and on servers through **Node.js**. It reads and changes the web page, talks to servers, and handles clicks. That power is why the most common web attack, **cross-site scripting (XSS)**, is simply "an attacker's JavaScript running inside someone else's page".

Try it now: open any browser, press **F12**, open the **Console** tab.

```
let user = "alice";            // variable that can change
const port = 443;             // variable that cannot be reassigned
console.log(`User ${user} on port ${port}`);

const ports = [22, 80, 443];
ports.forEach(p => console.log(p));          // arrow function

function isWeb(p) {
  return p === 80 || p === 443;              // === compares value and type
}
console.log(isWeb(443));                     // true

const person = { name: "alice", roles: ["admin", "dev"] };   // object, like a dict
console.log(person.roles[0]);
```

Talking to the page (the DOM)

The **DOM** is the page as a tree of objects that JavaScript can read and modify.

```
const box = document.querySelector("#output");   // find an element
box.textContent = "Hello";                        // set plain text
document.querySelector("button").addEventListener("click", () => {
  alert("clicked");
});
```

Talking to servers

```
fetch("https://api.github.com/users/octocat")
  .then(r => r.json())                  // parse the JSON body
  .then(d => console.log(d.login));     // octocat
```

With Node.js 18 or newer you can run the same file: `node script.js`.

Browser security rules you must know

- **Same-origin policy:** a page may read data only from its own origin (scheme + host + port). One site's script cannot read another site's responses.
- **CORS:** a server can relax that rule with `Access-Control-Allow-Origin`. A wildcard on a site that uses cookies, or a server that reflects any origin, is a misconfiguration.
- **Cookies:** `document.cookie` returns cookies that are not marked `HttpOnly`. That flag (Chapter 1) stops injected scripts from stealing the session.
- **`localStorage`:** any script on the page can read it. Storing login tokens there means one XSS bug exposes every user.

Attacker view: XSS in one line

```
out.innerHTML = userInput;     // if userInput is  <img src=x onerror=alert(1)>  the
script runs
```

`innerHTML` turns text into live HTML. An attacker who controls that text runs code with the victim's session: reading page data, sending requests as the user, redirecting to a fake login. Chapter 5 covers the types of XSS.

Defender view

```
out.textContent = userInput;   // treated as plain text, never executed
```

- Use `textContent`, not `innerHTML`, for untrusted text. Never use `eval()`, `new Function()` or `document.write()` with user data.
- Send a **Content-Security-Policy** header and set cookies `HttpOnly; Secure; SameSite`.
- Use `JSON.parse()` for JSON, never `eval()`.
- Scripts loaded from other sites run with your page's rights. Load only from sources you trust and pin versions.

5. Data formats: JSON and XML

Concept

Security tools talk to each other, to APIs and to your SIEM in structured text. Two formats cover most of it.

JSON

JSON (JavaScript Object Notation) is lightweight and everywhere: web APIs, Elastic and Wazuh logs, cloud audit logs, threat-intelligence feeds, configuration files.

```
{
  "timestamp": "2026-10-22T10:15:03Z",
  "src_ip": "203.0.113.50",
  "user": "admin",
  "event": "login_failed",
  "count": 5,
  "tags": ["ssh", "bruteforce"],
  "geo": { "country": "NL", "city": null }
}
```

Rules: keys are double-quoted strings. Values are string, number, `true` / `false`, `null`, array `[ ]` or object `{ }`. No comments, no trailing commas.

```
import json
data = json.loads(open("event.json").read())     # text -> Python dict
print(data["src_ip"], data["geo"]["country"])
print(json.dumps(data, indent=2))                # Python dict -> pretty text
```

Many log tools write **one JSON object per line** (called NDJSON or JSON Lines). The command-line tool `jq` filters JSON the way `grep` filters text:

```
sudo apt install jq
jq '.src_ip' event.json                                          # one field
jq -r 'select(.event=="login_failed") | .src_ip' events.ndjson   # filter, print raw text
jq -r 'select(.event=="login_failed") | .src_ip' events.ndjson | sort | uniq -c | sort -nr
```

XML

XML (Extensible Markup Language) uses tags and attributes. It is older and wordier, and still appears in SAML single sign-on, SOAP services, Windows event logs, Office documents, Android manifests, SVG images, and the **Nmap** scan output option `-oX`.

```
<?xml version="1.0"?>
<host>
  <address addr="192.168.56.101"/>
  <ports>
    <port protocol="tcp" portid="22">
      <state state="open"/>
      <service name="ssh"/>
    </port>
    <port protocol="tcp" portid="80">
      <state state="closed"/>
    </port>
  </ports>
</host>
```

Rules: every opening tag needs a closing tag (or self-closes with `/>`), tags nest properly, attribute values are quoted.

```
import xml.etree.ElementTree as ET
root = ET.parse("scan.xml").getroot()
for port in root.iter("port"):
    state = port.find("state").get("state")
    service = port.find("service")
    print(port.get("portid"), state, service.get("name") if service is not None else "-")
```

JSON vs XML

	JSON	XML
Size and readability	Compact, easy to read	Verbose, tag heavy
Data types	Strings, numbers, booleans, null, arrays	Everything is text unless a schema says otherwise
Comments	Not allowed	Allowed
Typical use	REST APIs, logs, cloud, threat feeds	SAML, SOAP, Windows events, Nmap, Office files
Main security risk	Trusting unvalidated fields, unsafe deserialization elsewhere	XXE (below), entity expansion denial of service

Attacker view

- **XXE (XML External Entity):** XML can declare entities that point to local files or URLs. A server whose parser resolves them can be tricked into returning `/etc/passwd` or making internal requests. SAML and SOAP endpoints are classic targets.
- **Entity expansion** (the "billion laughs" attack): tiny XML that expands to gigabytes and exhausts memory.
- **Injection through fields:** a username containing quotes, braces or script tags that the next system treats as code.
- **Secrets in JSON config files** pushed to public repositories.

Defender view

- Parse XML with a hardened parser: disable DTDs and external entities, or use the `defusedxml` library in Python.
- Validate input against a **schema** (JSON Schema, XSD): reject fields you did not expect.
- Treat every field in a log or API response as untrusted text. Escape it before it reaches a shell, a SQL query or a web page.
- Never commit tokens, keys or passwords. Add them to `.gitignore`, and scan your repository history for leaked secrets before you publish.

6. Secure coding rules in one table

The same few mistakes appear in every language. Learn these once.

Rule	Why	Python	Bash	JavaScript
Never trust input	All injection starts here	Validate with `ipaddress`, `re`, allow-lists	Check `$1` before use	Validate on the server, not just in the browser
Do not build commands from text	Command injection	`subprocess.run([...])`, no `shell=True`	Quote every variable	Avoid `child_process.exec` with user data
No `eval` on data	Runs attacker code	No `eval` / `exec` / `pickle` on untrusted data	Avoid `eval`	No `eval`, use `JSON.parse`
Escape output	Stops XSS and similar	Template auto-escaping	n/a	`textContent`, not `innerHTML`
Keep secrets out of code	Repos and history leak	`os.environ[...]`	`${VAR:?}` from environment	Server-side only, never in front-end code
Least privilege	Limits damage	Run as a normal user	Do not run scripts as root without need	Minimal API scopes and permissions
Handle errors quietly	Stack traces help attackers	Catch and log, show a generic message	`set -euo pipefail` and clear exit codes	Generic error pages
Keep dependencies patched	Libraries have CVEs	`pip list --outdated`	n/a	`npm audit`

7. Hands-on labs

Do these in your VM. Save each script and a short write-up (goal, code, output, what you learned) in your repo.

Lab 1: Python environment and hello tool

```
mkdir ~/codelab && cd ~/codelab
python3 -m venv venv && source venv/bin/activate
pip install requests
python3 -c "import requests; print(requests.__version__)"
```

Goal: a working venv. Write `hello.py` that asks for your name and a port number, then prints whether the port is a well-known one (0-1023).

Lab 2: Port checker

Save the port checker from section 2 as `portcheck.py`. Run it against `127.0.0.1`. Start a listener in another terminal (`python3 -m http.server 8080`) and extend the script to check port 8080.

Goal: see 8080 flip from closed to open. Capture one run in Wireshark or `tcpdump` and find the handshake from Chapter 1.

Lab 3: Failed-login report

Save the failed-login counter as `failed_ssh.py`. Generate failures (three wrong logins from your host, as in the Chapter 2 lab) and run `sudo python3 failed_ssh.py`. Then add a threshold: print `ALERT` when one IP has more than 5 failures.

Goal: your own host's IP appears at the top of the list, and the alert triggers once you cross the threshold.

Lab 4: Hash and verify

```
python3 hash.py ~/Downloads/yourfile.iso
sha256sum ~/Downloads/yourfile.iso       # must match
```

Goal: the two values match. Change one byte of a copy (`echo x >> copy.iso`) and watch the hash change completely.

Lab 5: Command injection, then the fix

Create `ping_bad.py` with the unsafe `os.system` code from section 2. Run it and enter `127.0.0.1; echo INJECTED`. Observe that both commands run. Then write `ping_good.py` with the `ipaddress` and `subprocess.run` version and enter the same text.

Goal: the unsafe version prints `INJECTED`; the safe version refuses the input. Explain why in two sentences.

Lab 6: Bash tools

Write `check_log.sh` and `sweep.sh` from section 3. Run `sweep.sh` only against your own host-only or lab network. Run `shellcheck` on both and fix every warning.

Goal: sweep lists your VM and host. `shellcheck` reports nothing.

Lab 7: XSS on a local page

Save as `xss-lab.html` and open it in your browser (file stays on your machine):

```
<!doctype html>
<meta charset="utf-8">
<input id="in" size="50" value="&lt;img src=x onerror=alert('XSS')&gt;">
<button onclick="unsafe()">Render UNSAFE</button>
<button onclick="safe()">Render SAFE</button>
<div id="out"></div>
<script>
  const out = document.getElementById("out");
  const text = () => document.getElementById("in").value;
  function unsafe() { out.innerHTML = text(); }
  function safe()   { out.textContent = text(); }
</script>
```

Goal: the first button pops an alert; the second shows the text harmlessly. In DevTools, open the Elements tab and compare what each button did to the page. Then run `document.cookie` in the console.

Lab 8: JSON and XML

```
cat > events.ndjson <<'EOF'
{"ts":"2026-10-22T10:15:03Z","src_ip":"203.0.113.50","user":"admin","event":"login_failed"}
{"ts":"2026-10-22T10:15:09Z","src_ip":"203.0.113.50","user":"root","event":"login_failed"}
{"ts":"2026-10-22T10:16:40Z","src_ip":"198.51.100.7","user":"alice","event":"login_ok"}
{"ts":"2026-10-22T10:17:02Z","src_ip":"203.0.113.50","user":"test","event":"login_failed"}
EOF
jq -r 'select(.event=="login_failed") | .src_ip' events.ndjson | sort | uniq -c | sort -nr

nmap -oX scan.xml 127.0.0.1
pip install defusedxml
```

Parse `scan.xml` with the ElementTree code from section 5, then switch the import to `defusedxml.ElementTree` and confirm it works the same.

Goal: `203.0.113.50` ranks first with 3 failures, and your script lists the open ports from the Nmap XML.

> **Next step on TryHackMe:** work through a Python or Bash scripting room, and write each solution up as a short script with comments in your repo.

8. Quick revision sheet

Syntax at a glance

Task	Python	Bash	JavaScript
Variable	`x = 5`	`x=5`	`let x = 5;`
Print	`print(x)`	`echo "$x"`	`console.log(x)`
If	`if x > 3:`	`if [[ $x -gt 3 ]]; then`	`if (x > 3) {`

Task	Python	Bash	JavaScript
Loop	`for i in range(3):`	`for i in 1 2 3; do`	`for (let i=0;i<3;i++) {`
Function	`def f(a):`	`f() { ... }`	`function f(a) { }`
List/array	`[1, 2, 3]`	`arr=(1 2 3)`	`[1, 2, 3]`
Dict/object	`{"a": 1}`	(use `declare -A`)	`{a: 1}`
Run a file	`python3 f.py`	`./f.sh`	`node f.js`

Useful commands

Task	Command
Create and enter a venv	`python3 -m venv venv && source venv/bin/activate`
Install a package	`pip install requests`
Lint a Bash script	`shellcheck script.sh`
Filter JSON lines	`jq -r 'select(.event=="x") | .src_ip' file`
Pretty-print JSON	`python3 -m json.tool file.json`
Check XML is well formed	`xmllint --noout file.xml`
Serve files locally	`python3 -m http.server 8080`

One-line summaries

- **Python** for tools and parsing. Use a venv. `subprocess.run([...])`, never `os.system` with user text.
- **Bash** for glue. Quote variables, `set -euo pipefail`, check `$?`.
- **JavaScript** runs in the browser. `textContent` is safe, `innerHTML` with untrusted text is XSS.
- **JSON** for APIs and logs, filter with `jq`. **XML** for SAML, Nmap and Windows events, parse with `defusedxml`.
- **Never trust input. Never `eval` data. Never commit secrets.**

9. Self-check quiz

Answer without looking back, then open the key.

1. Why use a Python virtual environment?
2. What does `connect_ex` returning 0 mean in the port checker?
3. What is wrong with `os.system("ping -c 1 " + host)`, and what is the safer form?
4. In Bash, what does `$?` hold, and what does `set -euo pipefail` do?
5. Why quote `"$dir"` in `rm -rf "$dir/tmp"`?
6. What is the difference between `innerHTML` and `textContent` for untrusted text?
7. Which cookie flag stops JavaScript from reading a session cookie?
8. Name two places JSON is common in security work.
9. What is XXE and how do you defend against it?
10. Why is it dangerous to store a login token in `localStorage`?

Show answer key

1. It keeps a project's packages separate from the system Python and from other projects, and avoids the `externally-managed-environment` error.
2. The TCP connection succeeded, meaning the port is open (the handshake completed).
3. The shell treats `;`, `&&` and `|` in the input as extra commands (command injection). Validate the input (for example with `ipaddress`) and use `subprocess.run([...])` as a list with no shell.
4. `$?` is the exit code of the last command (0 means success). `set -euo pipefail` stops the script on an error, on an unset variable, and when any stage of a pipe fails.
5. An empty or space-containing variable could change what is deleted. Quoting keeps it as one value, and `${dir:?}` stops the script when it is empty.
6. `innerHTML` parses text as live HTML and runs scripts attached to it (XSS). `textContent` always treats it as plain text.
7. `HttpOnly`.
8. Any two of: web APIs, SIEM and log data (Elastic, Wazuh), cloud audit logs, threat-intelligence feeds, configuration files.
9. XML External Entity: a parser that resolves external entities can be made to read local files or call internal URLs. Disable DTDs and external entities, or use `defusedxml`, and validate input against a schema.
10. Any script running on the page can read `localStorage`, so one XSS bug exposes the token. Prefer `HttpOnly` cookies.

What comes next

Chapter 4: Web Technologies. You can now read the code behind a page. Next you learn how the web is built end to end: HTML and CSS, JavaScript in context, cookies and sessions, and how web servers such as Apache and Nginx deliver it all. That is the ground you need before attacking or defending web applications in Chapter 5.